개발 포트폴리오

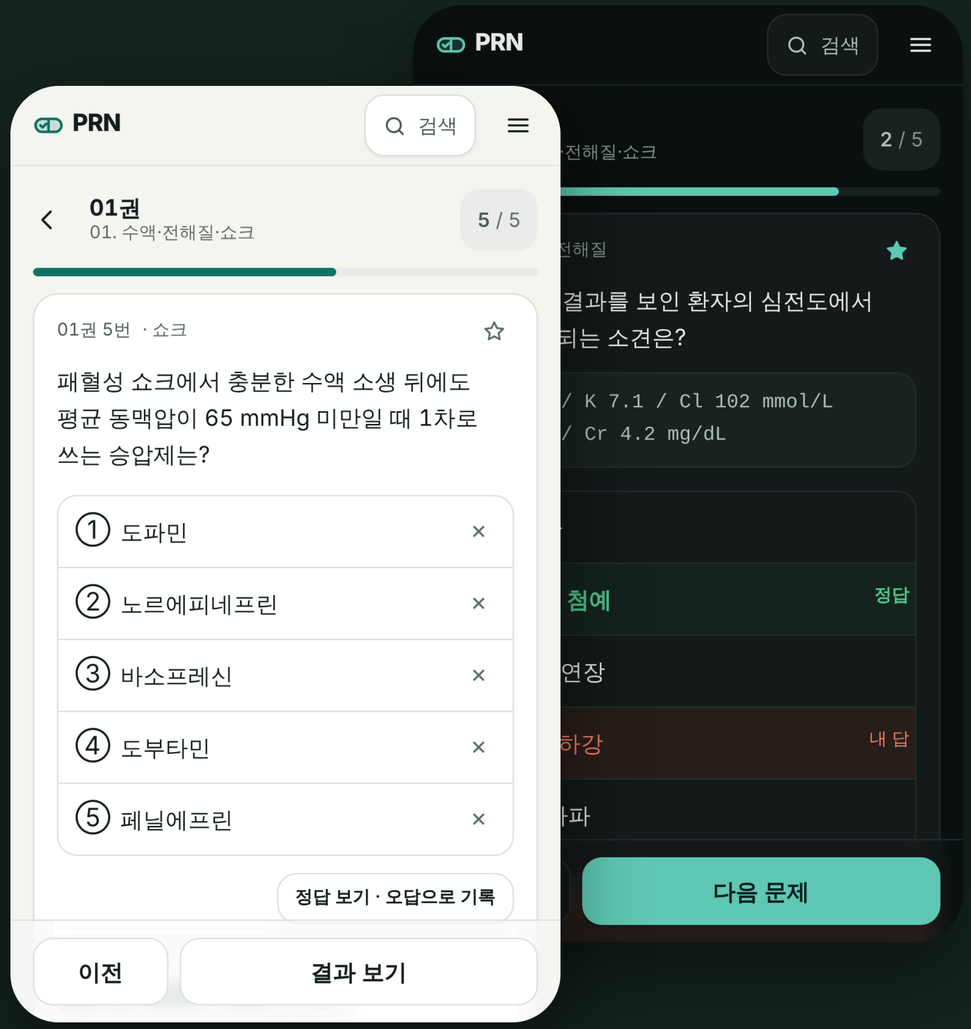
PRN

필요할 때 꺼내 쓰는 문제은행

PRN (*pro re nata*) — 처방전에서 「필요할 때마다」를 뜻하는 의학 약어

Next.js 14 · TypeScript · Supabase · PostgreSQL · Vercel · PWA

최성민 · firstsm41@naver.com · github.com/firstsm41



프로젝트 개요

혼자 기획하고, 만들고, 운영한 서비스

역할	기획 · DB 설계 · 프론트/백엔드 · 배포 · 운영
기간	2026.09 – 2026.11 · 버전 1.x → 2.6
사용자	외과 시험을 준비하는 학생과 문항을 검수하는 관리자
스택	Next.js 14 App Router · TypeScript · Tailwind · Supabase(Postgres · Auth · Storage) · Vercel · GitHub Actions · Python

1,600+
문항 · 챕터 20개

27
멱등 마이그레이션

16
테이블 전부 RLS

118
단위 테스트 + SQL · E2E

문제 정의

시험 자료는 많았지만, 쓸 수가 없었다

자료가 흩어져 있다

형성평가 스캔본, CBT 복원 PDF, 시험 사이트 HTML, 엑셀 정답표가 제각각

폰에서 찾을 수 없다

스캔 이미지는 검색이 안 되고, 띄어쓰기·기호가 달라 같은 말도 못 찾는다

정답을 믿기 어렵다

복원본마다 정답이 다르고, 누가 언제 고쳤는지 기록이 남지 않는다

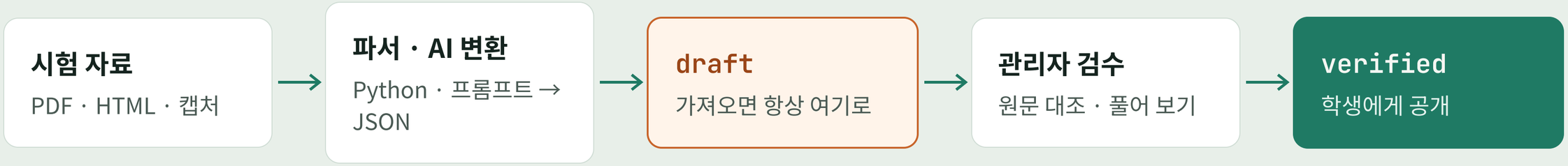
환자 사진이 섞여 있다

원본에 환자 사진이 있어 아무 곳에나 올리거나 공유할 수 없다

한 곳에 모으고, 검증하고, 필요할 때 어디서든 바로 꺼내 볼 수 있어야 한다

해결 방향

검수를 거친 문항만 학생에게 닿는다



풀기

한 화면에 한 문제. 답을 고르면 서버가 채점하고 바로 정답·해설

찾기

모든 화면 우상단에 고정된 검색창. 결과 카드 안에서 정답·해설까지

검수

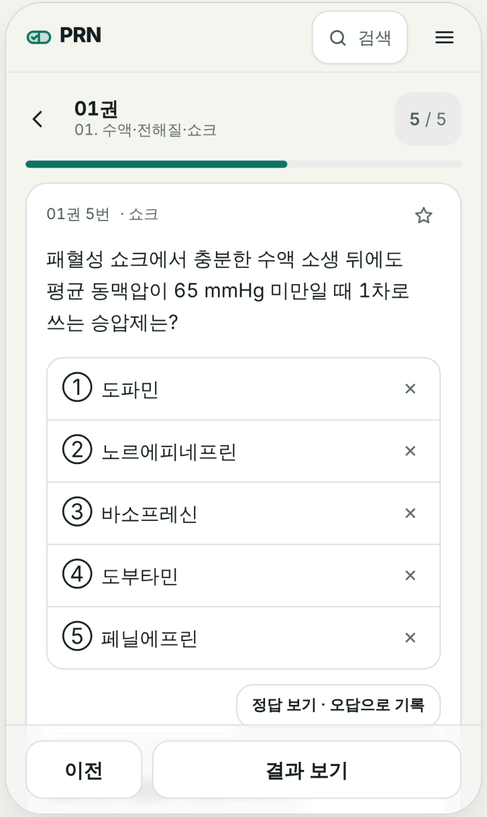
승인은 관리자만. 정답·해설 수정 이력은 DB 트리거가 자동 기록

학생 화면

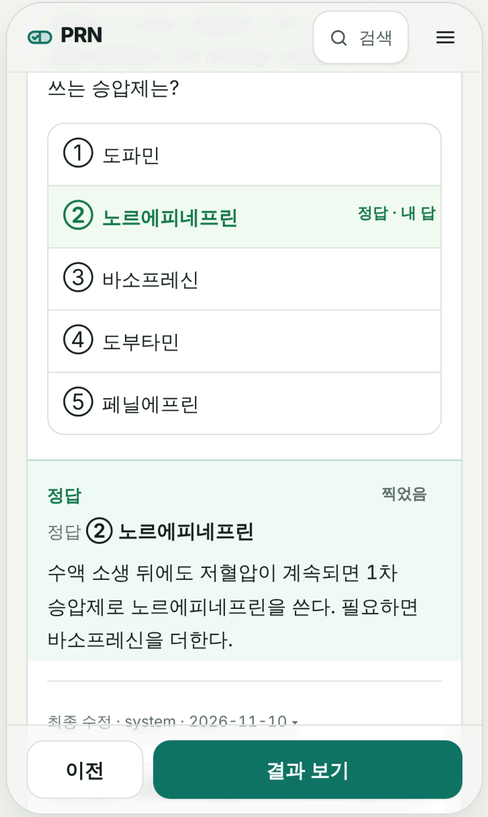
한 문제씩 풀고, 어디서든 바로 찾는다



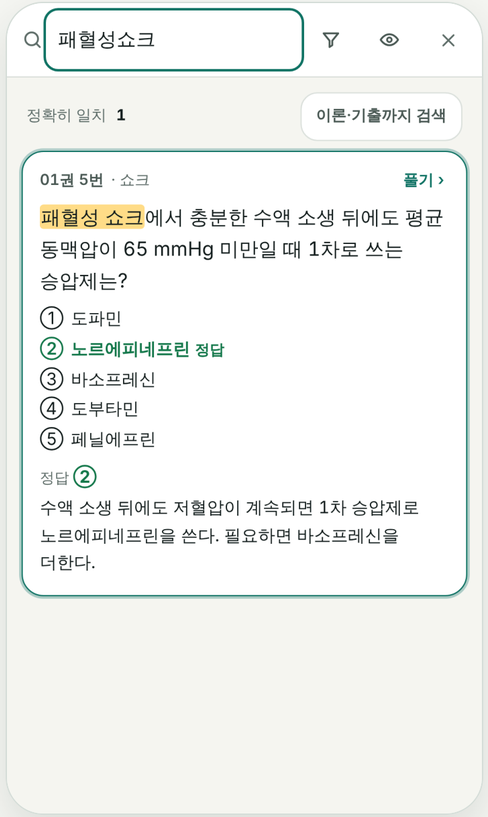
홈 · 진도와 D-day



한 화면에 한 문제



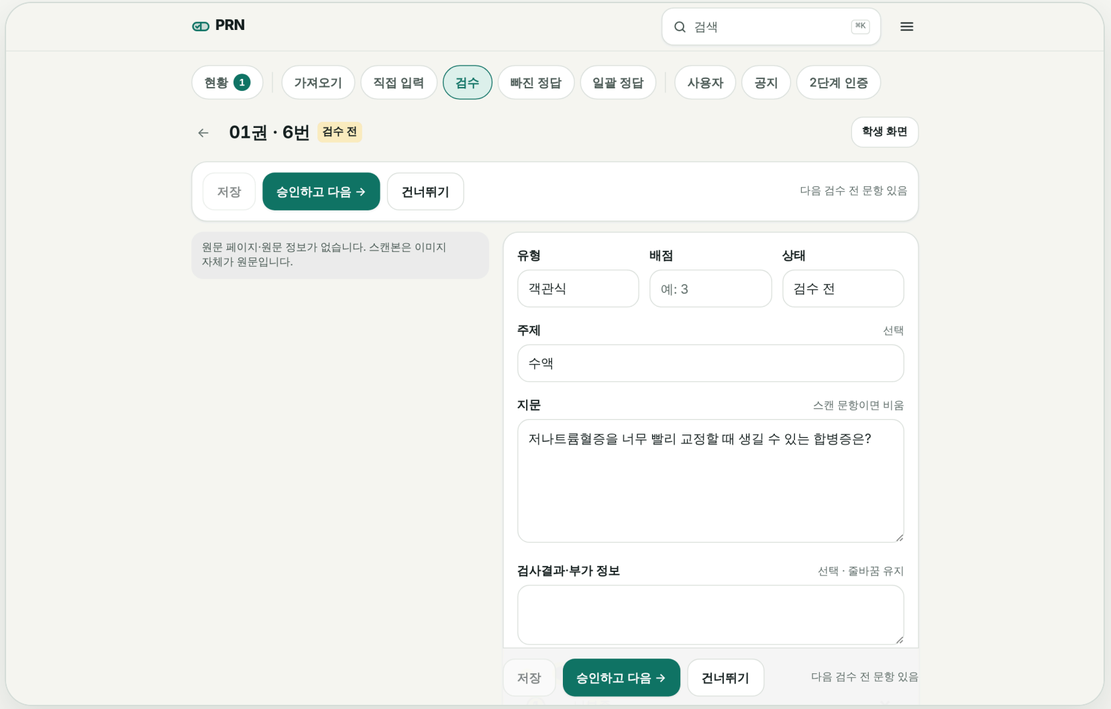
즉시 채점 · 해설



띄어쓰기 무시 검색

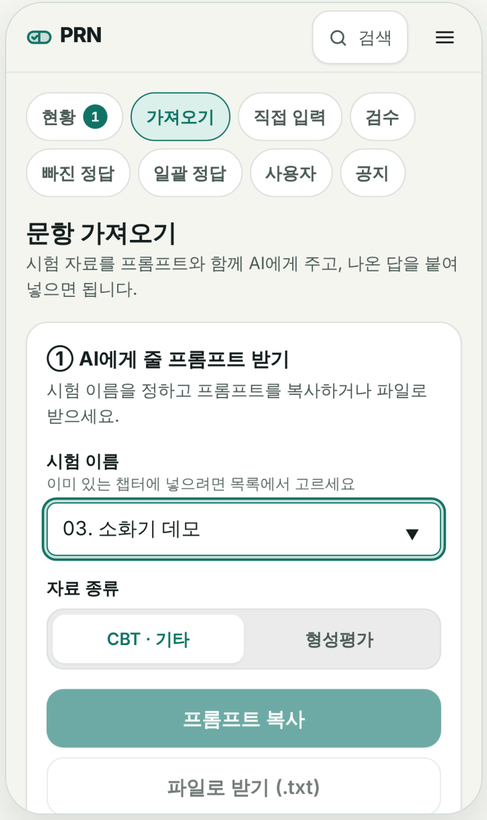
관리자 화면

가져오기 → 검수 → 승인을 한 흐름으로



PC 검수 편집기

원문 페이지 대조 · 사진 잘라내기 · 수정 이력과 되돌리기 · 「승인하고 다음」



AI로 문항 가져오기

풀면서 검토

정답을 가린 채 직접 풀어 본 뒤 승인

일괄 작업

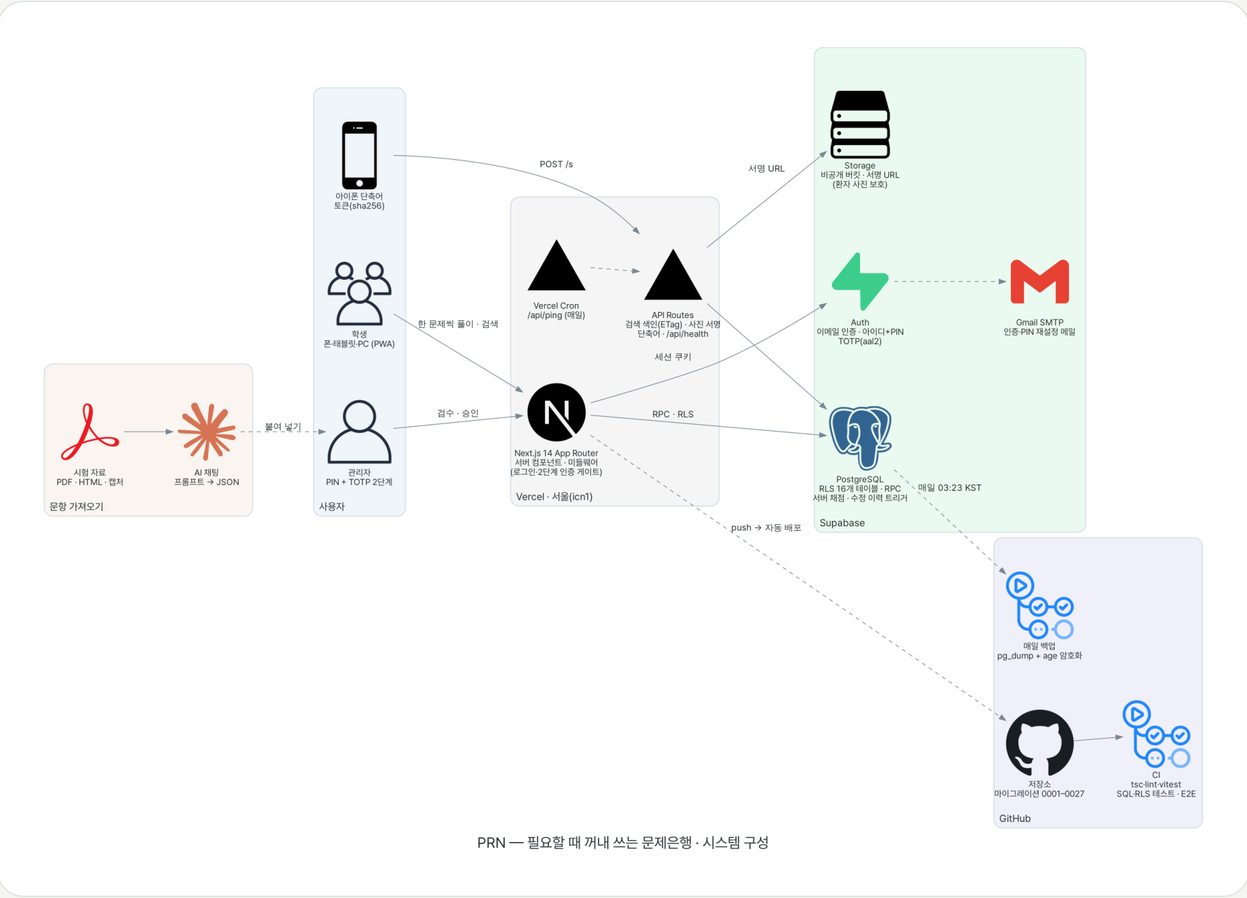
TSV 일괄 정답 · 일괄 승인, 묶음 단위 되돌리기

정답 의심

동기 절반 이상이 다른 답을 고른 문항

시스템 아키텍처

서버리스 + 관리형 Postgres, 운영비 0원



PRN — 필요할 때 꺼내 쓰는 문제은행 · 시스템 구성

Vercel 서울 리전

Next.js 서버 컴포넌트가 화면 데이터를 RPC로 가져온다. 미들웨어는 로그인·2단계 인증만 확인

Supabase

권한 판단은 Postgres RLS와 RPC. 사진은 비공개 버킷에 두고 짧은 서명 URL로만

GitHub Actions

PR마다 타입·린트·단위·SQL·E2E 검사, 매일 암호화 백업

애플리케이션 레이어

데이터는 서버에서, 권한은 DB에서

브라우저 PWA	**클라이언트 컴포넌트** 풀이·검색·편집기 — 채점·별표 같은 쓰기만	**IndexedDB 검색 색인** ETag/304로 갱신, 두 번째부터 즉시 검색
Next.js App Router	**`middleware.ts`** 세션 갱신 · 로그인/2단계 게이트	**서버 컴포넌트** 화면 데이터를 RPC로 한 번에
	Route Handlers 색인 · 서명 URL · 단축어 · 진단	
`src/lib` 도메인 로직	**채점 · 검색 · 정규화 · 가져오기** 테스트(*.test.ts)와 나란히, 타입은 types.ts 한 곳	**`supabase/{server,client}`** DB 클라이언트 생성은 여기서만
Supabase Postgres	**RPC** SECURITY DEFINER · 서버 채점	**테이블 16개 · RLS** 학생은 verified만 읽는다
	트리거 · Storage 수정 이력 자동 기록 · 비공개 버킷	

기술적 도전 1

한국어 검색은 정규화가 전부다

01 형태소 분석 대신 정규화

Postgres엔 한국어 분석기가 없다. 글자·숫자만 남긴 search_norm 열에 pg_trgm 색인

02 같은 규칙이 두 곳에

서버 검색은 SQL normalize_kr(), 오프라인 검색은 브라우저의 normalize.ts

03 1,800여 글자에서 결과가 달랐다

Postgres [[:punct:]]는 glibc 로캘 표, JS \p{P}는 유니코드 표를 따른다

04 모든 코드 포인트를 대조해 예외 표 생성

생성기가 Postgres에 전 유니코드를 넣어 비교 → 두 구현이 완전히 일치. 규칙이 바뀌면 캐시 버전을 올린다

normalize("...")

"패혈성 쇼크"
→ 패혈성쇼크

"①패혈성-쇼크"
→ 패혈성쇼크

"Na 138 / K 7.1"
→ na138k71

SQL과 TS가 같은 문자열을 내야 「패혈성 쇼크」로 「패혈성쇼크」를 찾는다

기술적 도전 2

보안은 앱이 아니라 DB가 지킨다

위협	대응
학생이 검수 전 문항·정답 원문을 보기	RLS — 학생은 verified만, 원문 페이지는 관리자만 읽는다
환자 사진 유출	비공개 Storage 버킷 + 짧은 서명 URL. 원본 자료는 저장소에 커밋하지 않는다
관리자 PIN 유출	TOTP 2단계 인증 — is_admin()이 aal2 세션을 요구한다
클라이언트 조작으로 정답률 부풀리기	서버 채점 RPC(record_attempt) — 앱이 정답 여부를 정하지 않는다
단축어 토큰 유출	sha256 해시만 저장 · 기기 고정 · 5개 상한 · 이용 정지 시 즉시 폐기
열린 가입으로 무료 DB 채우기	글 길이·이벤트 수 제한, 정지된 계정은 쓰기 불가

정답·해설 수정 이력은 트리거가 기록한다 — 앱 버그로 이력이 빠질 수 없다

품질과 운영

Supabase 없이도 끝까지 테스트한다

118

단위 테스트

채점 · 정규화 · 정답 파싱 · 가져오기 변환 (vitest)

×2

SQL 실행 테스트

빈 Postgres에 마이그레이션을 두 번 적용해 멱등을 확인하고, 학생·관리자·익명으로 RLS·RPC 검사

E2E

브라우저 테스트

Playwright가 로그인부터 풀이·검색·검수·승인·되돌리기까지 실제로 클릭

로컬 하네스 — Postgres + PostgREST + 직접 만든 가짜 Auth·Storage 게이트웨이(Node 약 200줄)로 Supabase를 흉내 낸다. 진짜 마이그레이션과 RLS 위에서 돈다

CI · PR마다 위 검사 전부

백업 · pg_dump + age 암호화

잠들기 방지 · 매일 쓰기 핑

자가 진단 · /api/health

회고

배운 것, 그리고 다음

배운 것

규칙을 먼저 글로 정한다

「한 문제씩」「검색창 고정」「정답 없어도 동작」 같은 절대 규칙 7개가 재작업을 줄였다

구현이 둘이면 반드시 어긋난다

사람의 주의 대신 생성기와 대조 테스트로 묶는다

앱에 버그가 있다고 가정한다

권한·채점·이력은 마지막 층인 DB가 지키게 설계

다음 단계

백업 다듬기

DB는 주 1회로 줄이고, 지금 빠져 있는 Storage 사진도 백업

인증 강화

관리자 2단계 인증 필수화, 가입·로그인에 CAPTCHA와 시도 제한

프레임워크 업그레이드

Next.js 15+로 올려 남은 보안 권고 해소